Starter

1. Define domain and co-domain
2. What are the four ways a first class object can be used?
3. Explain how partial function application is used to evaluate the expression: `multiply 4 7`
4. Write a function in Haskell code called `multiThree` that has 3 integer arguments and outputs the value of the arguments multiplied together

Functional Programming 2

Lists and Higher Order Functions

Lists in Haskell

Lists in functional programs aren't defined as a lot of elements, but rather a 'head' and 'tail'

Eg:

```
C#:       [1,2,3,4,5]
Haskell:  [1,2,3,4,5] ⇒ 1 :  [2,3,4,5]
```

There are a few functions you need to know:

head, tail, null

Lists in Haskell

Lists in functional programs aren't defined as a lot of elements, but rather a 'head' and 'tail'

Eg:

```
C#:       [1,2,3,4,5]
Haskell:  [1,2,3,4,5] ⇒ 1 :  [2,3,4,5]
```

There are a few functions you need to know:

head, tail, null We're going to think about what they might do, then I'll get you to write down the proper definitions.

Haskell `head`

What do we think ‘head’ is doing?

1. `head [1,2,3,4]`

2. `head [4,10,9]`

3. `head [[2,3],4,5,6]`

4. `head (head [[2,3],4,5,6])`

Haskell head

What do we think ‘head’ is doing?

1. `head [1,2,3,4]`

   `= 1`

2. `head [4,10,9]`

3. `head [[2,3],4,5,6]`

4. `head (head [[2,3],4,5,6])`

Haskell head

What do we think ‘head’ is doing?

1. `head [1,2,3,4]`

   `= 1`

2. `head [4,10,9]`

   `= 4`

3. `head [[2,3],4,5,6]`

4. `head (head [[2,3],4,5,6])`

Haskell head

What do we think ‘head’ is doing?

1. `head [1,2,3,4]`

   `= 1`
2. `head [4,10,9]`

   `= 4`
3. `head [[2,3],4,5,6]`

   `= [2,3]`
4. `head (head [[2,3],4,5,6])`

Haskell head

What do we think ‘head’ is doing?

1. `head [1,2,3,4]`

   `= 1`
2. `head [4,10,9]`

   `= 4`
3. `head [[2,3],4,5,6]`

   `= [2,3]`
4. `head (head [[2,3],4,5,6])`

   `= 2`

head

The haskell function ‘`head`’ returns the **first element** of the list.

[Warning, this may be a list!]

Haskell `tail`

What do we think ‘tail’ is doing?

1. `tail [1,2,3,4]`

2. `tail [4,10]`

3. `tail [[2,3],4,5,6]`

4. `tail (tail [[2,3],4,5,6])`

Haskell `tail`

What do we think ‘tail’ is doing?

1. `tail [1,2,3,4]`
   `= [2,3,4]`
2. `tail [4,10]`

3. `tail [[2,3],4,5,6]`

4. `tail (tail [[2,3],4,5,6])`

Haskell `tail`

What do we think ‘tail’ is doing?

1. `tail [1,2,3,4]`
   `= [2,3,4]`
2. `tail [4,10]`
   `= [10]`
3. `tail [[2,3],4,5,6]`

4. `tail (tail [[2,3],4,5,6])`

Haskell `tail`

What do we think ‘tail’ is doing?

1. `tail [1,2,3,4]`
   `= [2,3,4]`
2. `tail [4,10]`
   `= [10]`
3. `tail [[2,3],4,5,6]`
   `= [4,5,6]`
4. `tail (tail [[2,3],4,5,6])`

Haskell `tail`

What do we think ‘tail’ is doing?

1. `tail [1,2,3,4]`
   `= [2,3,4]`
2. `tail [4,10]`
   `= [10]`
3. `tail [[2,3],4,5,6]`
   `= [4,5,6]`
4. `tail (tail [[2,3],4,5,6])`
   `= [5,6]`

tail

The haskell function ‘`tail`’ returns the **everything after** the first element of the list.

[Warning, **will be** a list, even if it’s one item!]

Haskell `null`

What do we think ‘null’ is doing?

1. `null [1,2,3,4]`

2. `null []`

3. `null ( head [[2,3]] )`

4. `null ( tail [[2,3]] )`

Haskell `null`

What do we think ‘null’ is doing?

1. `null [1,2,3,4]`
   `= false`
2. `null []`

3. `null ( head [[2,3]] )`

4. `null ( tail [[2,3]] )`

Haskell `null`

What do we think ‘null’ is doing?

1. `null [1,2,3,4]`
   `= false`
2. `null []`
   `= true`
3. `null ( head [[2,3]] )`

4. `null ( tail [[2,3]] )`

Haskell `null`

What do we think ‘null’ is doing?

1. `null [1,2,3,4]`
   `= false`
2. `null []`
   `= true`
3. `null ( head [[2,3]] )`
   `= false`
4. `null ( tail [[2,3]] )`

Haskell `null`

What do we think ‘null’ is doing?

```
1. null [1,2,3,4]
       = false
2. null []
       = true
3. null ( head [[2,3]] )
       = false
4. null ( tail [[2,3]] )
       = true
```

null

The haskell function ‘`null`’ returns the true if the list **is empty** (ie []), and false if the list has elements.

Additional Tools

There are three additional things you'll see.
What do you think they'll do?

```
length x
```

```
value : list
```

```
listA ++ listB
```

Additional Tools

There are three additional things you'll see.
What do you think they'll do?

```
length x
```

→ the number of elements in the list

```
value : list
```

```
listA ++ listB
```

Additional Tools

There are three additional things you'll see.
What do you think they'll do?

`length x`
→ the number of elements in the list

`value : list`
→ prepends the value to the list. Note the similarity between this and the `head:tail` notation.

`listA ++ listB`

Additional Tools

There are three additional things you'll see.
What do you think they'll do?

`length x`
→ the number of elements in the list

`value : list`
→ prepends the value to the list. Note the similarity between this and the `head:tail` notation.

`listA ++ listB`
→ appends listB to listA

What would the following code output?

For list `xs = [3,5,2,8]`

1. `length xs`
2. `head xs`
3. `tail xs`
4. `6:xs`
5. `xs ++ [2]`
6. `xs`

What would the following code output?

For list `xs = [3,5,2,8]`

1. `length xs` 4
2. `head xs`
3. `tail xs`
4. `6:xs`
5. `xs ++ [2]`
6. `xs`

What would the following code output?

For list `xs = [3,5,2,8]`

1. `length xs` 4
2. `head xs` 3
3. `tail xs`
4. `6:xs`
5. `xs ++ [2]`
6. `xs`

What would the following code output?

For list `xs = [3,5,2,8]`

1. `length xs` 4
2. `head xs` 3
3. `tail xs` `[5,2,8]`
4. `6:xs`
5. `xs ++ [2]`
6. `xs`

What would the following code output?

For list `xs = [3,5,2,8]`

1. `length xs` `4`
2. `head xs` `3`
3. `tail xs` `[5,2,8]`
4. `6:xs` `[6,3,5,2,8]`
5. `xs ++ [2]`
6. `xs`

What would the following code output?

For list `xs = [3,5,2,8]`

1. `length xs` `4`
2. `head xs` `3`
3. `tail xs` `[5,2,8]`
4. `6:xs` `[6,3,5,2,8]`
5. `xs ++ [2]` `[3,5,2,8,2]`
6. `xs`

What would the following code output?

For list `xs = [3,5,2,8]`

1. `length xs` `4`
2. `head xs` `3`
3. `tail xs` `[5,2,8]`
4. `6:xs` `[6,3,5,2,8]`
5. `xs ++ [2]` `[3,5,2,8,2]`
6. `xs` `[3,5,2,8]`

What does this function do?

```
getItem (x:[]) = x
getItem (x:xs) = getItem xs
```

Show the steps of execution of `getItem` applied to the list `[1,2,3]`

What does this function do?

```
sumup [] = 0
sumup (x:xs) = x + sumup (xs)
```

Show the steps of execution of `sumup` applied to the list `[1,2,3]`

What does this function do?

```
sumup [] = 0
sumup (x:xs) = x + sumup (xs)

  = 1 + sumup [2,3]
  = 1 + 2 + sumup [3]
  = 1 + 2 + 3 + sumup []
  = 1 + 2 + 3 + 0
  = 6
```

Practice time!

On BPCompSci:

W201 – Haskell – Functional Programming with Lists

Higher Order Functions

A higher order function is a function that takes a function as an argument, or returns a function (or possibly both).

Higher Order Functions

A higher order function is a function that takes a function as an argument, or returns a function (or possibly both).

Higher order functions make it possible to define very general functions as well as some very complex behaviour.

Higher Order Functions

A higher order function is a function that takes a function as an argument, or returns a function (or possibly both).

Higher order functions make it possible to define very general functions as well as some very complex behaviour.

As functional programs are **stateless**, these high order functions are invaluable for parallel processing with **Big Data**.

Higher Order Functions

There are several that we need to know for the exam, and several of them have C# parallels!

Note: These aren't all of them, and the C# versions aren't necessary for your exams. It's just interesting 😀

Haskell	C#
`map`	`Select`
`filter`	`Where`
`foldl`	`Aggregate`
`foldr`	

`map`

`map` accepts a function to apply and a list to apply it to.

map

`map` accepts a function to apply and a list to apply it to.

EG:

```
square x = x * x

map square [1,4,-5,8]
```

map

`map` accepts a function to apply and a list to apply it to.

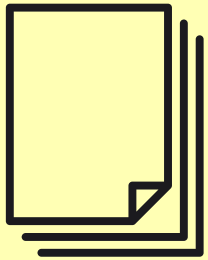
EG:

```
square x = x * x

map square [1,4,-5,8]
```

Returns `[1, 16, 25, 64]`

map



`map` accepts a function to apply and a list to apply it to.

EG:

```
square x = x * x
```

```
map square [1,4,-5,8]
```

Returns `[1, 16, 25, 64]`

What do the following evaluate to?

1. `calculation x = (x * 2) + 23`
   `map calculation [12, 34, 100]`
2. `map minimum  [[1, 3],[2, 7, 5],[9, 6]]`

Solutions:

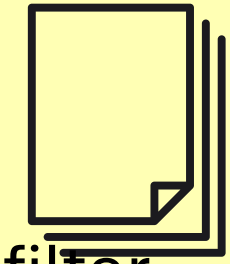
1. ```
   calculation x = (x * 2) + 23
   map calculation [12, 34, 100]

   = [47, 91, 223]
   ```

2. ```
   map minimum  [[1, 3],[2, 7, 5],[9, 6]]

   = [1, 2, 6]
   ```

`filter`



`filter` is a higher order function that uses a criterion to filter a list. Only elements that meet the criteria are kept.

`filter`

`filter` is a higher order function that uses a criterion to filter a list. Only elements that meet the criteria are kept.

EG:`filter odd [1, 2, 3, 4, 5, 6, 7]`

`filter`

`filter` is a higher order function that uses a criterion to filter a list. Only elements that meet the criteria are kept.

EG:`filter odd [1, 2, 3, 4, 5, 6, 7]`

Returns `[1, 3, 5, 7]`

`filter`

`filter` is a higher order function that uses a criterion to filter a list. Only elements that meet the criteria are kept.

EG:`filter odd [1, 2, 3, 4, 5, 6, 7]`

Returns `[1, 3, 5, 7]`

What do the following evaluate to?

1. `filter (>5) [1, 2, 3, 4, 5, 6, 7, 8, 9]`
2. `filter even [2, 4, 6, 11, 32]`

Solutions:

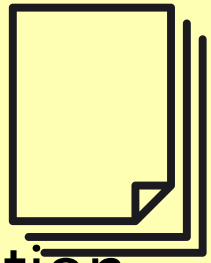
1. `filter (>5) [1, 2, 3, 4, 5, 6, 7, 8, 9]`

   `= [6, 7, 8, 9]`

2. `filter even [2, 4, 6, 11, 32]`

   `= [2, 4, 6, 32]`

`fold`



Fold is a process by which individual items from a collection are applied onto a **seed** using a provided function.

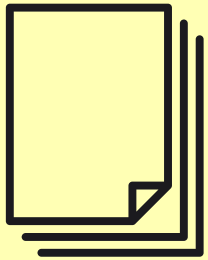
`fold`

Fold is a process by which individual items from a collection are applied onto a **seed** using a provided function.

Haskell has two forms, `foldl` and `foldr`. Your exam only has `fold`, and it is the same as `foldl`.

Note: That's 'fold left' and 'fold right'. `foldl` starts from the 'left' side, `foldr` starts from the 'right' side.

`foldl`



The higher order function takes a **combining function**, a **seed** and a **collection**.

EG: `foldl (+) 0 [1, 3, 5, 5]`

foldl

The higher order function takes a **combining function**, a **seed** and a **collection**.

EG: `foldl (+) 0 [1, 3, 5, 5]`

```
 = foldl (+) 1 [3, 5, 5]
 = foldl (+) 4 [5, 5]
 = foldl (+) 9 [5]
 = foldl (+) 14 []
 = 14
```

`foldr`

This performs as `foldl`, but starts at the end of the list, rather than the head.

EG: `foldr (+) 0 [1, 3, 5, 5]`

foldr

This performs as `foldl`, but starts at the end of the list, rather than the head.

EG: `foldr (+) 0 [1, 3, 5, 5]`

```
 = foldr (+) 5 [1, 3, 5]
 = foldr (+) 10 [1, 3]
 = foldr (+) 13 [1]
 = foldr (+) 14 []
 = 14
```

You Try

What do the following Haskell statements evaluate to?

1. `filter even [1, 2, 3, 4, 5, 6, 7, 8, 9]`
2.
```
div6 x = (mod x 6) == 0
filter div6 [12, 5, 23, 18, 100, 36]
```
3. `map abs [-1, 2, -3]`
4. `foldl (-) 1 [10, 12, 14]`
5. `foldr (++) "!" ["World", "Hello"]`

Solutions:

1. `filter even [1, 2, 3, 4, 5, 6, 7, 8, 9]`
   `= [2, 4, 6, 8]`
2. `div6 x = (mod x 6) == 0`
   `filter div6 [12, 5, 23, 18, 100, 36]`
   `= [12, 18, 36]`
3. `map abs [-1, 2, -3]`
   `= [1, 2, 3]`
4. `foldl (-) 1 [10, 12, 14]`
   `= -35`
5. `foldr (++) "!" ["World", "Hello"]`
   `= "!HelloWorld"`

Practice Time!

On BPCompSci.com

W202 - Haskell - Functional Programming with High Order Functions